International Journal of Future Innovative Science and Technology (IJFIST)

|ISSN: 2454-194X | A Bimonthly, Peer-Reviewed, Scholarly Journal |

Volume 9, Issue 3, May-June 2026

DOI: 10.15662/IJFIST.2026.0903006

Why Agentic AI Initiatives Keep Failing Before Production: An Architect's Perspective on Autonomous Operations

Viswanathan Tiruchindurai Srinivasan

Associate Vice President, Technology Consulting, Persistent Systems, Texas, USA

viswanathan_t@persistent.com

Publication History: Received: 04.05.2026; Revised: 24.05.2026; Accepted: 28.05. 2026; Published: 06.06.2026.

ABSTRACT: Enterprises are increasingly deploying Agentic AI -systems that don't just predict but act -inside live business operations, yet most of these initiatives stall before reaching sustained production use. This article argues that the failure is architectural and organizational, not a data science shortfall: proofs of concept are built in lab conditions that do not resemble production, autonomy is granted faster than trust is earned, and guardrail, governance, and change-management work is treated as optional polish rather than a prerequisite. Drawing on enterprise architecture practice, the article proposes a risk-tiered autonomy model, a concrete guardrail architecture, a four-step implementation roadmap (knowledge base, retrieval-augmented generation pipeline, model testing, integration and deployment), and a consolidated checklist that together convert Agentic AI adoption from an unpredictable initiative into a repeatable engineering and governance discipline.

KEYWORDS: Agentic AI, Enterprise Architecture, MLOps, Retrieval-Augmented Generation (RAG), AI Governance, Autonomous Operations, Guardrails, Change Management

I. INTRODUCTION

Enterprises have run predictive and generative AI in production for years; forecasting demand, scoring risk, summarizing documents, answering customer questions. The current wave of Agentic AI is different in kind, not just degree: these are systems that don't just predict but act opening tickets, adjusting configurations, approving transactions, rerouting work, or triggering rollback with limited or no human review of each individual step. This class of initiative is failing to reach sustained production use at a rate that should concern every technology and operations leader.

This is not a data science problem. It is an architecture, governance, and operating-model problem, magnified by the fact that the systems being automated are often live, business-critical, and subject to regulatory or contractual obligations. A recommendation engine that is wrong is an inconvenience. An agent that is wrong and has write access to a production system, a financial transaction, or a customer record is an incident.

This article examines why AI and Agentic AI initiatives stall before reaching durable production use and lays out a concrete set of architectural and organizational steps that reduce that failure rate -applicable across operational domains, whether the system being automated is a customer service desk, an IT infrastructure environment, a supply chain, a finance back office, or any other operational function.

Part 1: The Common Failure Pattern

AI as a lab experiment, not an operating capability. Most AI initiatives begin inside an innovation lab, a data science center of excellence, or a vendor-sponsored proof of concept -often disconnected from the operational team that will eventually have to own the outcome. The model performs well on curated historical data, gets a demo slot, but does not get an operational owner, a support budget, or a rollback plan. The pattern repeats because incentives are misaligned from day one:

- The lab is measured on model accuracy and pilot count, not on the operational metric the business actually cares about (cycle time, cost per transaction, error rate, resolution time).

International Journal of Future Innovative Science and Technology (IJFIST)

|ISSN: 2454-194X | A Bimonthly, Peer-Reviewed, Scholarly Journal |

Volume 9, Issue 3, May-June 2026

DOI: 10.15662/IJFIST.2026.0903006

- The operations team is measured on service levels and reliability, and treats an unproven automation layer as additional risk, not additional capacity.
- No one owns the P&L line the AI initiative is supposed to move.

A production-grade use case needs, at minimum, a named owner inside the operating function, a quantified target tied to an existing operational metric and baselined before the initiative starts, and a roadmap that treats the model or agent as one component of a larger operating capability -including monitoring, escalation, retraining, and eventual decommissioning.

The gap between the lab environment and the live environment. Proofs of concept are almost always built against a sanitized, static, and partial view of the business -a curated data extract, a batch of historical cases, or a simulated environment. Production looks nothing like that: streaming data from multiple systems with different formats and quality levels; event storms during incidents or peak periods; systems of record that were never designed for real-time API access; and overlapping process variants and exceptions accumulated over years of organizational change. An agent grounded on curated data behaves unpredictably the moment it meets that distribution shift, which is why pilots that "worked great" in the lab often produce a wave of false positives once connected to live systems.

The architectural fix is two explicit lanes: an **experimentation lane** (sandboxed, replayed, or synthetic data, no write access to live systems) and a **production lane** (live data through governed, versioned interfaces, with explicit read-only, recommend-only, and closed-loop stages). Promotion between lanes should be a documented, repeatable checklist -performance thresholds, shadow-mode results, rollback tested, runbook signed off -not an ad hoc decision made under pressure to show a demo.

Part 2: Why Agentic AI Fails Differently Than Predictive AI

Predictive and generative AI initiatives -where a system plans and executes actions with limited human review -fail for an additional, more serious reason: **the blast radius of an autonomous action in a live environment is fundamentally different from the blast radius of a wrong prediction.**

The autonomy spectrum. Automation maturity is commonly described as: **L0** manual operations; **L1/L2** assisted or narrowly scoped, pre-approved actions; **L3** conditional autonomy with human supervision and override; **L4** high autonomy with exception-based escalation; **L5** full, closed-loop autonomy. Most initiatives fail because they try to leap from L0/L1 directly to an L4-like demo without building the L2/L3 scaffolding -bounded action catalogs, approval workflows, audit trails, and a supervised track record. The correct posture is deliberately incremental: each domain should progress L1 → L2 → L3, with autonomy earned through a measured track record rather than assumed from a successful pilot.

The missing guardrail layer. An agent that can query data, reason over it, and call tools to execute actions needs a guardrail layer that most first-generation pilots simply do not build, because guardrails do not appear in a demo:

- **Action catalogs with declared blast radius** -every tool an agent can call needs a declared scope: which systems, which time windows, which magnitude of change is permitted.
- **Pre-execution simulation or dry-run** against a recent state snapshot, flagging deviations from expected behavior before a change is pushed live.
- **Rate limiting and circuit breakers**, so a runaway agent cannot issue an unbounded number of actions in a short window.
- **Human-in-the-loop escalation with a real approval SLA** -a defined approver, a defined response-time SLA, and a defined safe-state fallback if the approver does not respond.
- **Immutable audit and explainability trail** recording what data triggered an action, what reasoning path fired, what was called, and the outcome.
- **Deterministic rollback** for every action; if a change cannot be reliably reversed, it stays a recommend-only action.

Without this layer, initiatives either stay permanently in "recommend only" mode and eventually get defunded, or they ship into limited production and cause an incident that ends the program -often taking better-designed initiatives down with it in the resulting loss of organizational trust.

Grounding and hallucination risk. Where language models reason over operational data, correlate signals, or draft remediation plans, they can produce fluent, confident, and factually wrong statements. The engineering response: ground reasoning in live retrieval from systems of record rather than memorized context; constrain action generation to

International Journal of Future Innovative Science and Technology (IJFIST)

|ISSN: 2454-194X | A Bimonthly, Peer-Reviewed, Scholarly Journal |

Volume 9, Issue 3, May-June 2026

DOI: 10.15662/IJFIST.2026.0903006

a pre-validated catalog rather than free-form commands; architecturally separate the reasoning component from the execution component, with independent validation at execution time; and continuously evaluate diagnostic accuracy against a library of known-good historical cases, tracked as a standing operational metric rather than a one-time validation exercise.

Part 3: The Data and Integration Bottleneck

Why is operational data structurally harder than it looks. Enterprise operational data has characteristics that make AI initiatives materially harder than a first look suggests:

- **System heterogeneity** -a mix of legacy and modern systems, each with different data models and integration interfaces, requiring a substantial, ongoing normalization investment.
- **Volume and velocity** that make naive storage and processing approaches infeasible in high-transaction environments.
- **State and configuration drift**, since the operating environment changes constantly and a model grounded on last month's snapshot is reasoning about an environment that no longer fully exists.
- **Label scarcity**, since ground truth for "what actually caused this" typically exists only in unstructured case notes written under time pressure.

The shared infrastructure argument. Because these are structural, not one-off problems, they should be funded and built as shared data infrastructure -a normalized data/event bus, a curated feature store, a governed reference-data API, and a labeled case corpus -rather than rebuilt inside every individual use case. This infrastructure does not show a direct return on its own, which is why it is chronically underfunded relative to individual pilots that promise a visible win; the case has to be made at the portfolio level, since the cumulative time-to-value across ten or twenty use cases, not any single one, justifies the up-front investment.

Environment parity and MLOps/LLMOps. A model or agent that performs differently in staging than in production is a reliability issue, not an inconvenience. This requires dev, test, and production environments with genuinely equivalent data schemas and interface versions; CI/CD for models and agent policies with the rigor used for application code, including regression testing against historical case replay and staged rollout; continuous monitoring for data drift, concept drift, and agent behavior drift; and a defined retraining cadence, since a model trained before a major process change can silently degrade well before an obvious failure signal appears.

Part 4: Governance That Matches Operational Risk

Risk-tiering. Generic AI governance (privacy, bias, explainability) is necessary but insufficient for action-taking AI. A tier structure based on operational blast radius is needed: **Tier 1** read-only insight, no write access; **Tier 2** recommend with mandatory human execution; **Tier 3** bounded autonomous action with human oversight and override; **Tier 4** broader closed-loop autonomy, reserved for domains with a proven Tier 3 track record. Each tier should carry predefined requirements for testing rigor, approvals, and rollback readiness -decided once at the framework level, not negotiated per use case, which is what turns governance into an unpredictable, late-stage blocker.

Security, Risk, and Legal as design partners. The recurring governance failure is architectural review happening only at the end of a build cycle, forcing a costly redesign. The corrective pattern brings Security, Risk, and Legal into the design phase of any Tier 2+ initiative: Security reviews the action catalog and guardrail design before implementation begins; Legal assesses implications for customer-facing decisions, financial actions, and jurisdictional obligations early; Risk helps set consistent tiering thresholds across business units.

A Center of Excellence. A dedicated function, staffed jointly by the operating function, Data Science, Platform Engineering, and Security, improves production survival odds by maintaining the shared infrastructure described in Part 3, publishing and enforcing tiering standards, running a formal red-teaming and evaluation function, and acting as the point where lessons from one domain are carried into the next rather than re-learned from scratch.

Part 5: Change Management for the Operations Team

Why technically successful automation still gets rejected. An operator whose work is reduced to reviewing AI-generated recommendations experiences that change as a threat to expertise unless the transition is managed deliberately. The evidence is consistent: usage quietly declines, staff revert to manual processes, and the initiative is eventually reported as "adopted" in name only. Sponsors need honest answers before rollout: what does the role look like once an agent handles routine cases -eliminated, or elevated to complex-case handling and supervision? Are

|ISSN: 2454-194X | A Bimonthly, Peer-Reviewed, Scholarly Journal |
Volume 9, Issue 3, May-June 2026
DOI: 10.15662/IJFIST.2026.0903006

incentives aligned to the outcomes automation improves? Is there a defined career pathway toward supervising and tuning autonomous systems?

Building trust incrementally. Trust in an autonomous system is built the way trust in a new team member is built - through a track record on low-stakes tasks before higher-stakes responsibility:

- **Shadow mode as a mandatory phase**, where the agent runs in parallel with existing manual processes and its recommendations are logged and compared against what staff did before any promotion decision.
- **Transparent explanation of every recommendation**, so staff can verify reasoning, not just trust a black box -a much faster path to genuine adoption.
- **A visible override and feedback mechanism**, where a rejected recommendation is captured, categorized, and fed back into the evaluation function, so staff see their judgment actively shaping the system.

Part 6: Fragmented Ownership Across Silos

AI initiatives characteristically touch the business function, IT/Data Platform, Operations, Security, external vendors, and often a systems integrator. When no single role owns the initiative end-to-end, accountability diffuses and a familiar sequence follows: a vendor delivers a working proof of concept, IT struggles to integrate it with production systems, Operations was not consulted on the operating model and resists adoption, and the initiative stalls in permanent "integration in progress."

A more resilient pattern separates three roles explicitly: an **AI/Agentic AI Product Owner** inside the operating function, accountable to leadership for the business case and roadmap; a **standing cross-functional squad** combining domain expertise, data engineering, agent engineering, platform/DevOps, and security, with genuine decision rights over design trade-offs; and a **defined escalation path**, with a named decision owner and target resolution time, for the recurring trade-offs -speed versus control, experimentation versus standardization, vendor-proprietary versus open interfaces -that otherwise stall projects for weeks. The durable fix is treating AI as a recognized, permanently staffed product line, not a temporary program that disbands after initial rollout.

Part 7: Choosing the Right First Use Cases

Many AI programs launch with a mandate to "transform operations," and the first wave of use cases is either too trivial to matter or too ambitious for the organization's current data and governance maturity. A disciplined selection approach favors use cases that are simultaneously **well-bounded** -operating within a single domain with a clear, measurable outcome; **data-feasible today** -using data and case history that already exist in usable form, rather than requiring a multi-year foundation build; and **reversible** -so that early autonomous actions can be automatically and reliably rolled back, keeping the initial risk profile low while trust-building mechanisms mature.

Representative early candidates that consistently meet this bar include routine case triage and routing, redundant-alert or noise suppression so staff see root-cause signals instead of alert storms, and forecast-driven scheduling or resourcing. Each has a bounded, reversible action set and a low direct business-impact risk if imperfect -properties well suited to proving out the autonomy ladder before attempting higher-risk domains such as financial transactions. A phased discipline -a Minimum Viable Autonomous capability at Tier 2 or 3, expanded only as the track record justifies -builds the credibility that larger subsequent investments depend on.

Part 8: The Implementation Roadmap

Step 1 -Create an Enterprise Knowledge Base. Assemble a governed repository of standard operating procedures, system and integration documentation, historical case resolutions with root-cause analysis, reference and master data, metric definitions, and escalation/approval policies. Organize content by domain, tagging each item by system, process area, applicable version, and risk tier so retrieval can be scoped precisely. Version every procedural document explicitly -a stale procedure is not a minor inconvenience; it is the kind of error the guardrail architecture in Part 2 exists to catch downstream. Require SME sign-off before publication, build an explicit deprecation workflow for retired procedures, and build ingestion connectors to the systems where this knowledge already lives, with OCR support for legacy documents and deduplication across overlapping content from different teams.

Step 2 -Build an RAG Pipeline. Use structure-preserving chunking so procedures, tables, and step-by-step documentation remain complete, actionable units rather than arbitrary fragments. Pair dense vector search with keyword and metadata filtering, since exact identifiers and case numbers behave more like identifiers than natural language. Ground retrieval against live systems of record alongside the static knowledge base, so answers reflect the environment as it exists today. Re-rank retrieved content and cite the specific source document and version behind

|ISSN: 2454-194X | A Bimonthly, Peer-Reviewed, Scholarly Journal |
Volume 9, Issue 3, May-June 2026
DOI: 10.15662/IJFIST.2026.0903006

every recommendation, which is what makes the audit trail requirement enforceable. Set confidence thresholds that trigger escalation to a human rather than letting the model fill a gap with a plausible but ungrounded answer and evaluate retrieval precision and recall as a metric independent of generation quality.

Step 3 -Test Models. Build a golden evaluation set from historical cases with known outcomes for offline testing of diagnostic accuracy and hallucination rate before any live exposure. Run mandatory shadow-mode testing on live data with no execution authority, comparing the agent's output to actual staff decisions. Red-team the agent against event storms, malformed data, conflicting sources, and -where a language model is involved -attempts to manipulate its behavior through crafted case text. Test latency and throughput under realistic operational load, since many decisions have tight response-time windows. Require structured SME review and sign-off before any promotion to an action-taking tier.

Step 4 -Integration and Deployment. Connect the agent to core business systems, case-management, and reference-data systems through governed, versioned APIs, surfaced in the channels staff already use rather than a separate disconnected tool. Apply CI/CD to prompts, retrieval configuration, and action policies with the same rigor used for application code. Deploy through canary rollout -a limited team, region, or process variant first -before expanding, mirroring standard change-management practice. Route autonomous actions through the same change-approval and logging infrastructure used for manual changes, so both are visible in a single audit trail. Activate drift monitoring, rate limiting, and rollback as active production controls from day one, not a follow-up hardening exercise. Feed every new case and every staff correction back into the knowledge base, so the capability improves through operational use.

This four-step sequence is iterative rather than strictly linear: findings from Step 3 testing routinely surface gaps in the Step 1 knowledge base, and production feedback from Step 4 routinely sends teams back to the Step 2 retrieval configuration.

Part 9: A Consolidated Checklist to Prevent Failure

Ownership and business case

1. Assign a named AI/Agentic AI Product Owner inside the operating function before development begins.
2. Define the target operational metric and baseline it before the initiative starts.
3. Fund the initiative as an operating capability with an ongoing support and retraining budget, not a one-time project.

Architecture and data

4. Build or extend shared data, feature store, and reference-data API infrastructure rather than a bespoke pipeline per use case.
5. Establish an experimentation lane and a separate production lane, with a documented promotion checklist between them.
6. Ensure environment parity across dev, test, and production for data schemas, latency, and interface versions.
7. Implement CI/CD for models and agent policies, including regression testing against a historical case replay library.
8. Implement continuous monitoring for data drift, concept drift, and agent behavior drift.

Knowledge base and RAG pipeline

1. Stand up a governed enterprise knowledge base with SME sign-off and version control before any content is used for retrieval.
2. Build an RAG pipeline with structure-preserving chunking, hybrid retrieval, and live grounding against systems of record.
3. Require source citation on every generated recommendation, with confidence thresholds that trigger escalation.
4. Track retrieval precision and recall as a metric independent of generation quality.

Model testing and validation

1. Build a golden evaluation set from historical cases with known outcomes for offline testing.
2. Run every new agent in mandatory shadow mode against live data, comparing output to actual staff decisions.
3. Red-team the agent against event storms and adversarial input before granting autonomy above Tier 1.
4. Test latency and throughput under realistic operational load.
5. Require structured SME review and sign-off before promotion to any action-taking tier.

|ISSN: 2454-194X | A Bimonthly, Peer-Reviewed, Scholarly Journal |
Volume 9, Issue 3, May-June 2026
DOI: 10.15662/IJFIST.2026.0903006

Agent specific guardrails

1. Define an explicit action catalog per agent, with declared blast radius, reversibility, and rate limits.
2. Require pre-execution simulation or dry-run validation for any action above Tier 1 risk.
3. Build deterministic, tested rollback for every autonomous action before it is promoted out of shadow mode.
4. Separate the reasoning/planning component from the execution component architecturally.
5. Maintain an immutable audit trail of every action, its trigger, its reasoning path, and its outcome.

Governance

1. Adopt a risk-tiering framework and apply it consistently across all AI use cases.
2. Involve Security, Risk, and Legal as design partners from the start of any Tier 2+ initiative.
3. Stand up or engage an AI Center of Excellence to own standards, shared infrastructure, and evaluation.

Change management and adoption

1. Redesign operating roles and incentive structures around the post-automation workflow before rollout.
2. Provide transparent, staff-facing explanations for every recommendation, and capture override feedback.

Integration, deployment, and sequencing

1. Integrate agents through governed, versioned APIs, routed through existing change-approval and audit infrastructure.
2. Deploy through canary rollout before expanding, mirroring standard change-management practice.
3. Feed every new case and staff correction back into the knowledge base and evaluation set continuously.
4. Select initial use cases that are well-bounded, data-feasible today, and operate on reversible actions.
5. Advance autonomy tiers incrementally, based on a demonstrated track record at the current tier.
6. Treat the initiative as a permanent, staffed product line, not a temporary program that disbands after rollout.

II. CONCLUSION

Enterprise operating environments are unusually unforgiving for AI and Agentic AI initiatives, because the systems being optimized are often business-critical, customer-facing, and subject to regulatory or contractual obligations, and because the operational complexity of heterogeneous, multi-system environments makes the gap between a lab pilot and a production-ready capability wider than most teams expect. The initiatives that survive past the pilot stage share a common pattern: they are owned as operating capabilities rather than experiments, built on shared and governed data infrastructure rather than bespoke pipelines, equipped with an explicit guardrail and autonomy-tiering architecture rather than an all-or-nothing leap to closed-loop automation, and rolled out with as much deliberate investment in change management as in the underlying model or agent.

None of this eliminates the inherent difficulty of automating live business operations. What it does is convert that difficulty from an unpredictable, late-stage source of project failure into a set of known, manageable engineering and governance decisions -made deliberately, at the right point in the initiative, by the people who will ultimately be accountable for the system's behavior in production.

REFERENCES

1. National Institute of Standards and Technology (NIST). *AI Risk Management Framework (AI RMF 1.0)*. U.S. Department of Commerce, 2023.
2. International Organization for Standardization. *ISO/IEC 42001:2023 -Information Technology, Artificial Intelligence Management System*. ISO, 2023.
3. Lewis, P., Perez, E., Piktus, A., et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
4. Sculley, D., Holt, G., Golovin, D., et al. "Hidden Technical Debt in Machine Learning Systems." *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
5. Google Cloud Architecture Center. *MLOps: Continuous Delivery and Automation Pipelines in Machine Learning*. Google, 2023.
6. OWASP Foundation. *OWASP Top 10 for Large Language Model Applications*. OWASP, 2023.